
華中科技大學

課程實驗報告

課程名稱：C 語言程序設計實驗

專業班級：計算機科學與技術 2502

學 號：I202520033

姓 名：魏宇翔

指導教師：李平

報告日期：2025 年 12 月 21 日

計算機科學與技術學院

目 录

1 实验 6 指针程序设计实验.....	1
1.1 程序改错与跟踪调试	1
1.2 程序完善与修改替换	3
1.3 程序设计	9
1.4 小结	25
2 实验 7 结构与联合.....	26
2.1 表达式求值的程序验证	26
2.2 源程序修改替换	27
2.3 程序设计	31
2.4 小结.....	49
参考文献.....	50

1 实验 6 指针程序设计实验

1.1 程序改错与跟踪调试

我通过观察与跟踪调试代码发现代码存在如下图所示两处错误：

```
/* 实验6-1程序改错与跟踪题源程序 */
#include<stdio.h>
char * strcpy(char *, const char *);
int main(void)
{
    char *s1, *s2, *s3;
    printf("Input a string:\n");
    scanf("%s", s2);
    strcpy(s1, s2);
    printf("%s\n", s1);
    printf("Input a string again:\n");
    scanf("%s", s2);
    s3=strcpy(s1, s2)
    printf("%s\n", s3);
    return 0;
}

/* 将字符串s复制给字符串t,并且返回串t的首地址 */
char * strcpy(char *t, const char *s)
{
    while(*t++=*s++);
    return (t);
}
```

错误一：仅定义字符指针但没有初始化，使指针变成悬挂指针指向随机内存地址，会对读写操作造成影响。

解决方案：在定义指针 s1、s2、s3 后，我定义三个字符数组 a[30]、b[30]、c[30]，这些数组分配了固定大小的合法内存空间，再将指针 s1 绑定到数组 a 的首地址、s2 绑定到数组 b 的首地址、s3 绑定到数组 c 的首地址，使指针指向可合法读写的内存，消除悬挂指针带来的内存访问风险。

代码处理如下：

```
char *s1, *s2, *s3;
char a[30], b[30], c[30];
s1 = a; s2 = b; s3 = c;
```

错误二：strcpy 函数返回值错误，在这个函数里的 while(*t++=*s++) 循环执行时，指针 t 会持续自增，最终指向字符串后面的无效地址，这样会导致获取到错误的地址，从而打印错误。

解决方案：在函数内部增加变量 p 并赋值为 t 的初始地址，这样在循环过程中 t 仍正常自增完成字符串复制，最终返回保存的首地址 p，确保获取到正确的字符串首地址，保证后续打印操作能正常输出字符串内容。

代码处理如下：

```
char *strcpy(char *t, const char *s)
{
    char *p = t;
    while (*t++ = *s++);
    return (p);
}
```

我的源代码如下：

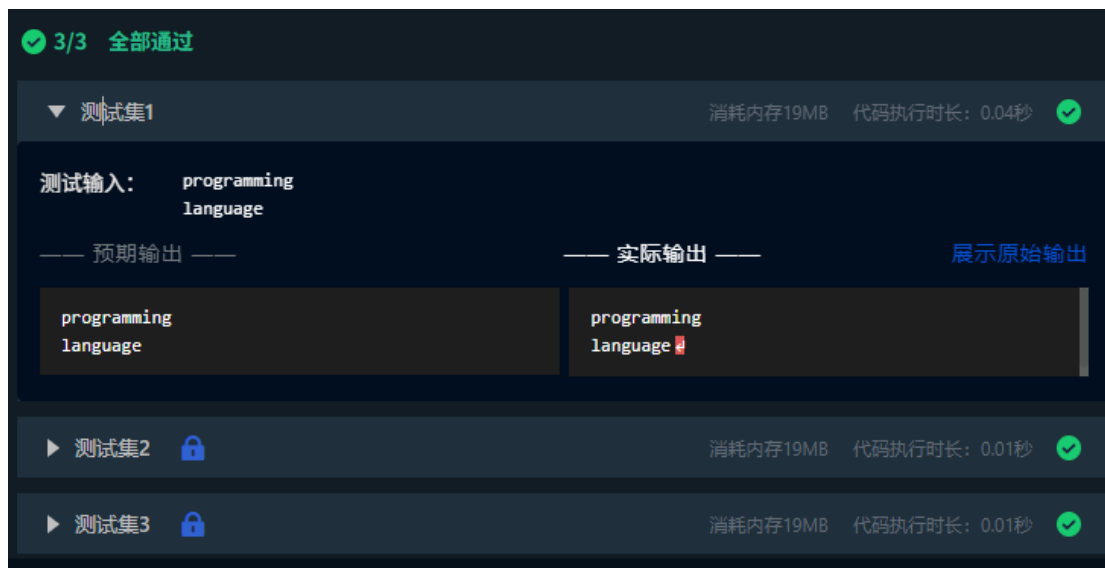
```
#include<stdio.h>
char *strcpy(char *t, const char *s);

int main(void)
{
    char *s1, *s2, *s3;
    char a[30], b[30], c[30];
    s1 = a; s2 = b; s3 = c;

    scanf("%s", s2);
    strcpy(s1, s2);
    printf("%s\n", s1);
    scanf("%s", s2);
    s3 = strcpy(s1, s2);
    printf("%s\n", s3);

    return 0;
}

char *strcpy(char *t, const char *s)
{
    char *p = t;
    while (*t++ = *s++);
    return (p);
}
```



最后程序成功编译并运行成功。

1.2 程序完善与修改替换

1. 程序完善和修改替换——字符串排序（填空）

本关要求填写合适代码完善程序，让 `strsort` 函数用于对字符串进行升序排序，再主函数中输入 N 个字符串存入通过 `malloc` 动态分配的存储空间，然后调用 `strsort` 函数对这 N 个串按字典升序排序。

完善后的代码如下：

```
/* 实验6程序完善与修改替换第（1）题源程序：字符串升序排序 */
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define N 100
/* 对指针数组s指向的size个字符串进行升序排序 */
void strsort ( char *s[ ],int size )
{
    char *temp;
    int i, j ;
    for(i=0; i<size-1; i++)
        for(j=0; j<size-i-1; j++)
            if (strcmp(s[j], s[j+1]) > 0)
            {
                temp=s[j];
                s[j] = s[j+1];
                s[j+1]=temp;
            }
}
```

编译运行结果如下：

3/3 全部通过

▼ 测试集1

消耗内存1023.1MB 代码执行时长：0.01秒

测试输入：

3
C
Python
Java

—— 预期输出 ——

—— 实际输出 ——

展示原始输出

C
Java
Python

C
Java
Python

▶ 测试集2

消耗内存1023.1MB 代码执行时长：0.01秒

▶ 测试集3

消耗内存1023.1MB 代码执行时长：0.01秒

2. 程序完善和修改替换——字符串排序（改进）

本关要求二级指针形参重写第 1 关的 `strsort` 函数，并且在该函数体的任何位置都不允许使用下标引用

我首先把排序函数 `strsort` 的形参从 `char s[]` 改成了 `char **s`，满足了题目要求不使用下标的方式操作，通过增加的 `p1`、`p2` 两个二级指针，靠 `p1++`、`p2++` 遍历指针数组，再用 `p1`、`*p2` 取得字符串地址来进行比较和交换，靠指针实现了冒泡排序；并且我还检查了 `malloc` 的返回值，只要要是内存分配失败，就把已经分配的内存都释放掉再退出程序，避免空指针出问题；最后输出排序结果后，把动态分配的内存全部释放掉，。

改进后的代码如下：

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAX_STR_LEN 50
#define MAX_NUM 100
void strsort(char**s,int size)
{
    char*temp;
    int i,j;
    char**p1,**p2;
    for(i=0;i<size-1;i++)
    {
        for(j=0,p1=s,p2=s+1;j<size-i-1;j++,p1++,p2++)
        {
            if(strcmp(*p1,*p2)>0)
            {
                temp=*p1;
                *p1=*p2;
                *p2=temp;
            }
        }
    }
}

```

```

int main()
{
    int n,i;
    char*str_arr[MAX_NUM];
    char temp_buf[MAX_STR_LEN];
    scanf("%d",&n);
    getchar();
    for(i=0;i<n;i++)
    {
        fgets(temp_buf,MAX_STR_LEN,stdin);
        temp_buf[strcspn(temp_buf,"\n")]='\0';
        str_arr[i]=(char*)malloc(strlen(temp_buf)+1);
        if(str_arr[i]==NULL)
        {
            printf("内存分配失败! \n");
            for(int k=0;k<i;k++)
                free(str_arr[k]);
            return 1;
        }
        strcpy(str_arr[i],temp_buf);
    }
}

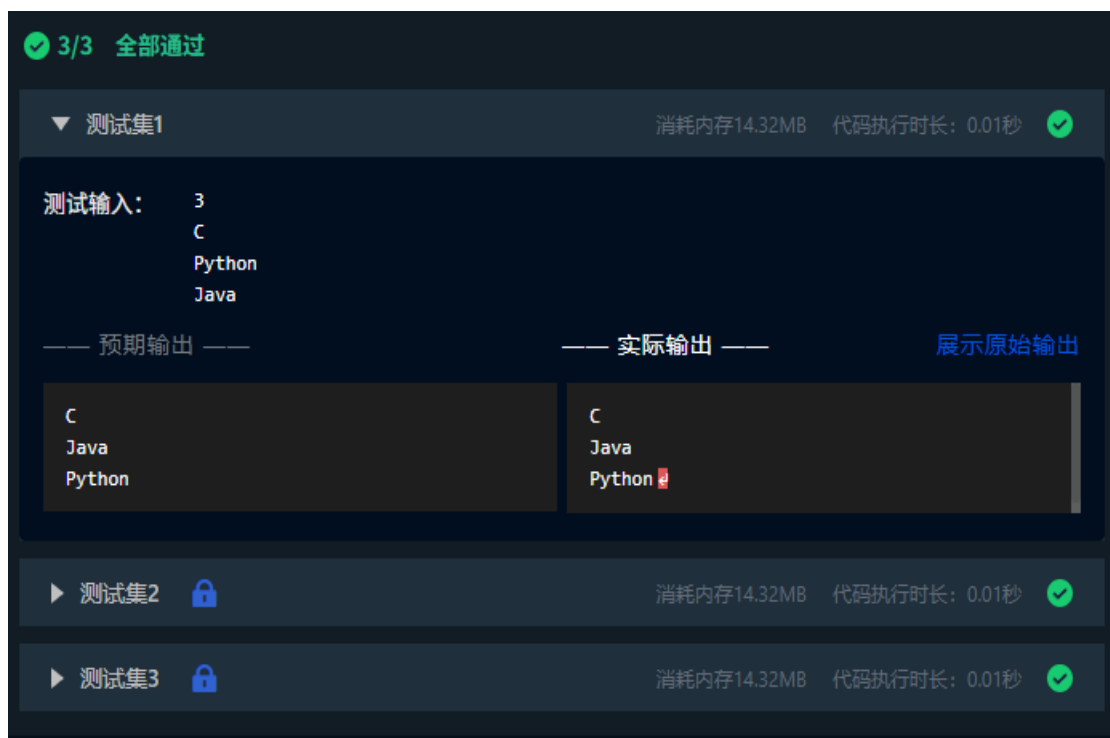
```

```

strsort(str_arr,n);
for(i=0;i<n;i++)
{
    puts(str_arr[i]);
    free(str_arr[i]);
    str_arr[i]=NULL;
}
return 0;
}

```

编译运行结果如下：



3. 程序完善和修改替换——指针函数（填空）

本关要求填写合适代码完善程序，通过函数指针和菜单选择来调用库函数实现字符串操作。

因为题目要求我们用菜单选择实现函数调用，所以我们要使用 switch 函数进行调度。

完善后的代码如下：


```

#include <stdio.h>
#include <string.h>

int main(void) {
    char *(*p)(char *, const char *);
    char a[100], b[100], *result;
    int choice;

    while (1) {
        scanf("%d", &choice);
        if (choice == 4) break;
        getchar();
        fgets(a, sizeof(a), stdin);
        a[strcspn(a, "\n")] = '\0';
        fgets(b, sizeof(b), stdin);
        b[strcspn(b, "\n")] = '\0';
        switch (choice) {
            case 1: p = strcpy; break;
            case 2: p = strcat; break;
            case 3: p = strtok; break;
        }
        result = p(a, b);
        printf("%s\n", result);
    }
    return 0;
}

```

编译运行结果如下

3/3 全部通过

测试集1

消耗内存20.12MB
 代码执行时长: 0.02秒

测试输入:

```

1
the more you learn,
the more you get.
2
the more you learn,
the more you get.
3
www.educoder.net
.
4

```

预期输出

```

the more you get.
the more you learn,the more you get.
www

```

实际输出

展示原始输出

```

the more you get.
the more you learn,the more you get.
www

```

▶ 测试集2

消耗内存20.12MB
 代码执行时长: 0.01秒

▶ 测试集3

消耗内存20.12MB
 代码执行时长: 0.01秒

4. 程序完善和修改替换——指针函数数组（改进）

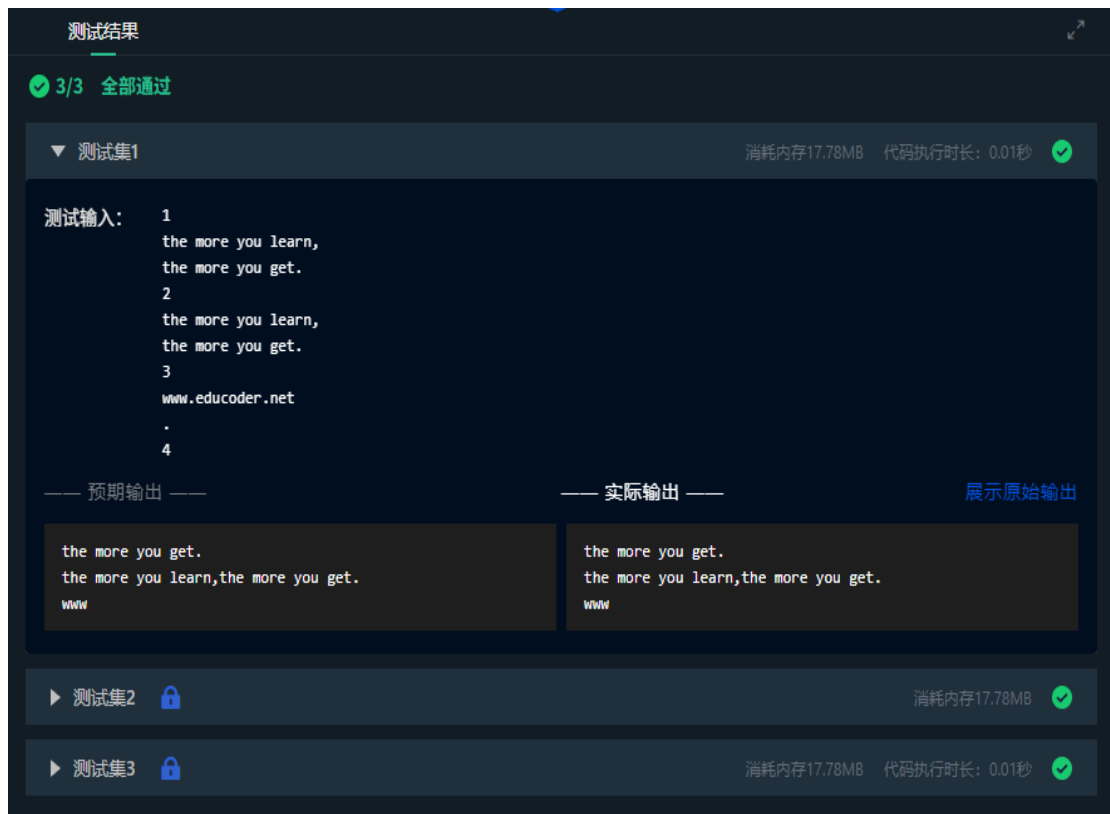
本关要求使用转移表而不是 switch 语句重写第 3 关程序。

转移表是一种数据结构，用于根据输入值确定需要执行的函数或操作。通常是一个包含指针的数组，数组中每个元素包含指向对应函数或操作的指针。于是我把需要用到的 strcpy、strcat、strtok 这三个字符串函数，全都放进 func_table 的数组里（即转移表），让数组和操作指令对应好。

改进后的代码如下：

```
#include<stdio.h>
#include<string.h>
int main(void){
    typedef char*(*StrFunc)(char*,const char*);
    StrFunc func_table[]={strcpy,strcat,strtok};
    char a[100],b[100],*result;
    int choice;
    while(1){
        scanf("%d",&choice);
        if(choice==4)break;
        getchar();
        fgets(a,sizeof(a),stdin);
        a[strcspn(a,"\n")]='\0';
        fgets(b,sizeof(b),stdin);
        b[strcspn(b,"\n")]='\0';
        result=func_table[choice-1](a,b);
        printf("%s\n",result);
    }
    return 0;
}
```

编译运行结果如下：



1.3 程序设计

1. 通过指针取出字节

本关要求编写一个程序，从整型变量的高字节开始，依次取出每字节的高 4 位和低 4 位并以十六进制数字字符的形式进行显示，要求通过指针取出每字节。

我思路是：先输入一个无符号整数存在 a 里，这样做位运算时补的都是 0，不会受符号位影响。然后用字符型指针 p 指向 a，这样就能一个字节一个字节地访问 a 了。最后从高字节到低字节依次取每个字节，把每个字节右移 4 位拿出高 4 位，再用按位与 0x0f 拿出低 4 位，以小写十六进制输出。

我的流程图如下图 1-1：

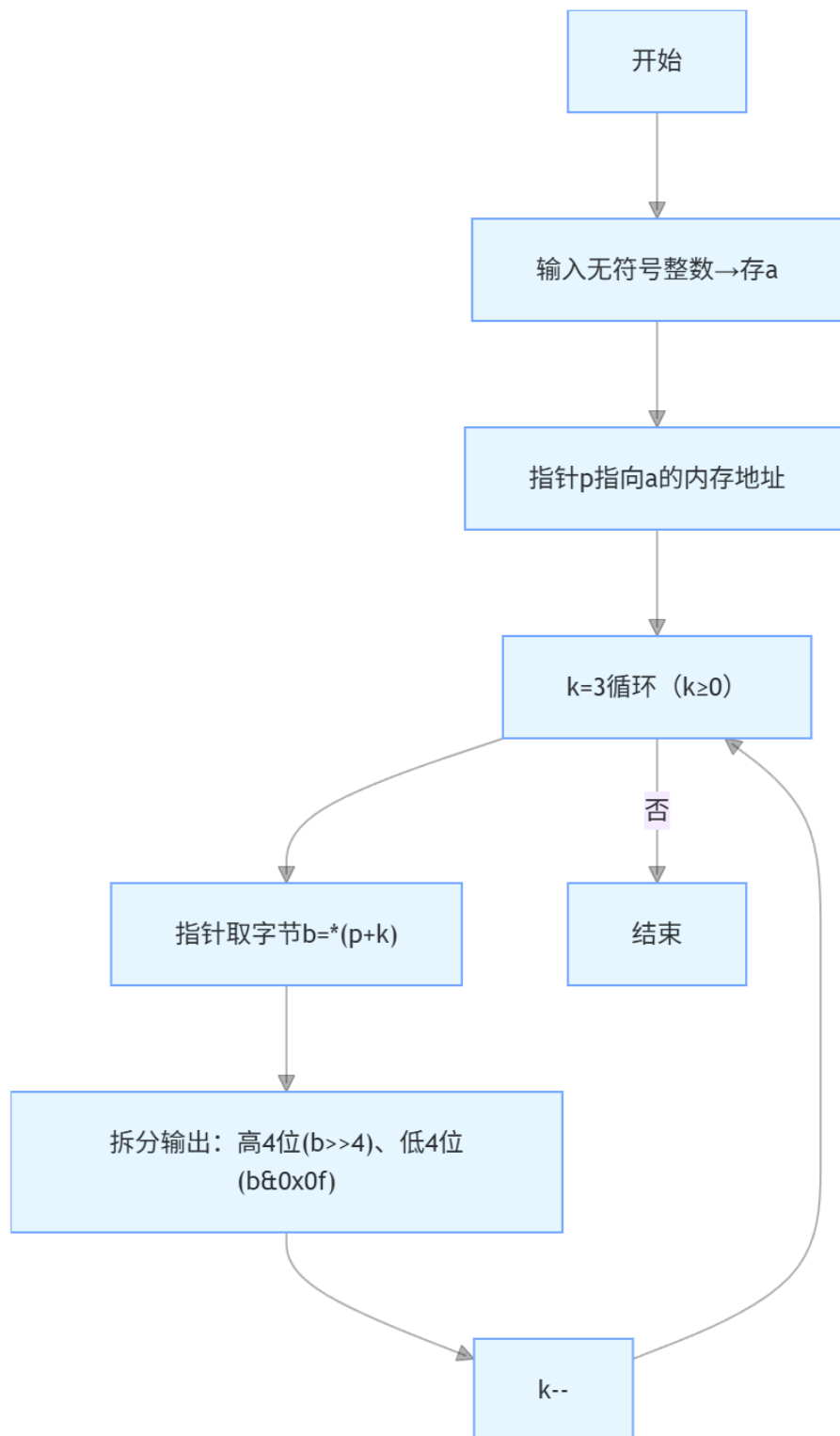
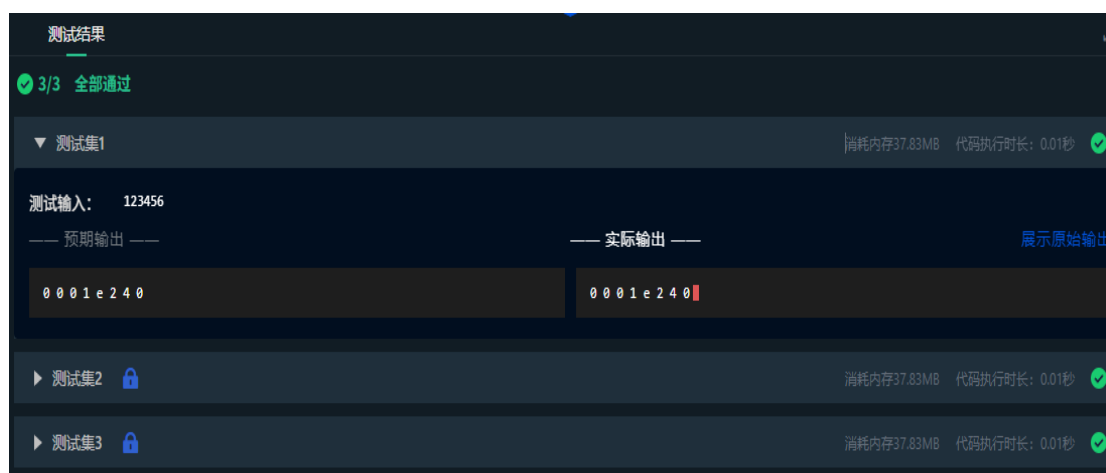


图 1-1 程序设计题 1.3.1 的流程图

我的源代码如下：

```
#include<stdio.h>
int main()
{
    unsigned long a;
    scanf("%lu",&a);
    unsigned char*p=(unsigned char*)&a;
    for(int k=3;k>=0;k--)
    {
        unsigned char b=*(p+k);
        printf("%x %x ",b>>4,b&0x0f);
    }
    return 0;
}
```



2. 删除重复元素

本关要求定义函数 RemoveSame(a, n)，去掉有 n 个元素的有序整数序列 a 中的重复元素，返回去重后序列的长度。

我的思路是：先在主函数中检验输入的数组长度，在读取有序整数数组元素后，调用 RemoveSame 函数，指针 p 标记去重后数组的最后一个有效元素，指针 q 遍历数组，如果 q 指向的元素与 p 不同，则将 q 的值赋给 p 的下一个位置，p 后移并更新新长度，跳过重复元素。最后主函数再通过指针遍历输出去重后的数组元素，并打印去重后的数组长度。

我的流程图如下图 1-2：

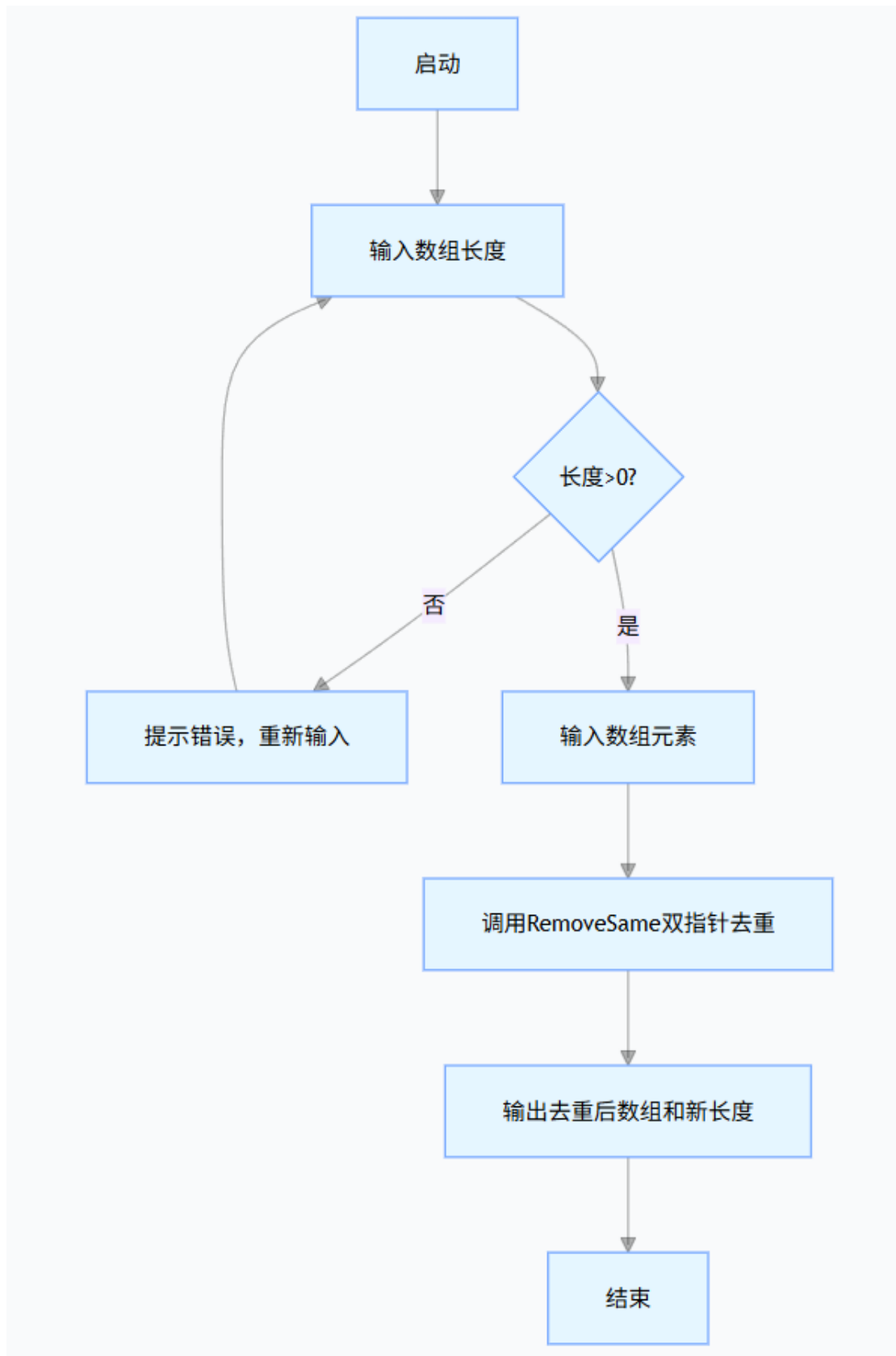


图 1-2 程序设计题 1.3.2 的流程图

我的源代码如下：

```
#include<stdio.h>

int RemoveSame(int*a,int n){
    if(n==0){
        return 0;
    }
    int*p=a;
    int*q=a+1;
    int new_len=1;
    while(q<a+n){
        if(*q!=*p){
            p++;
            *p=*q;
            new_len++;
        }
        q++;
    }
    return new_len;
}
```

```
int main(){
    int arr_size;
    do{
        scanf("%d",&arr_size);
        if(arr_size<=0){
            printf("输入错误！数组个数不能小于等于0，请重新输入。 \n");
        }
    }while(arr_size<=0);
    int nums[100];
    for(int i=0;i<arr_size;i++){
        scanf("%d",&nums[i]);
    }
    int*p_nums=nums;
    int new_length=RemoveSame(p_nums,arr_size);
    int*p=p_nums;
    while(p<p_nums+new_length){
        if(p==p_nums+new_length-1){
            printf("%d",*p);
        }else{
            printf("%d ",*p);
        }
        p++;
    }
    printf("\n%d",new_length);
    return 0;
}
```



3. 旋转图像

本关要求输入图像矩阵的行数 n 和列数 m , 接下来的 n 行每行输入 m 个整数, 表示输入的图像, 输出原始矩阵逆时针旋转 90° 后的矩阵。

我的思路是: 读取图像矩阵的行数 n 和列数 m , 接着读取 n 行 m 列的原始矩阵数据并存储到二维数组 `mat` 中; 然后利用双重循环遍历原始矩阵的每一个元素, 因为图像逆时针旋转 90° , 所以原始矩阵中第 i 行第 j 列的元素, 会对应到旋转后矩阵的第 $m-1-j$ 行第 i 列。将所有元素依次赋值到旋转后的数组 `rotated` 中; 最后按行遍历输出 `rotated` 数组。

我的流程图如下图 1-3:

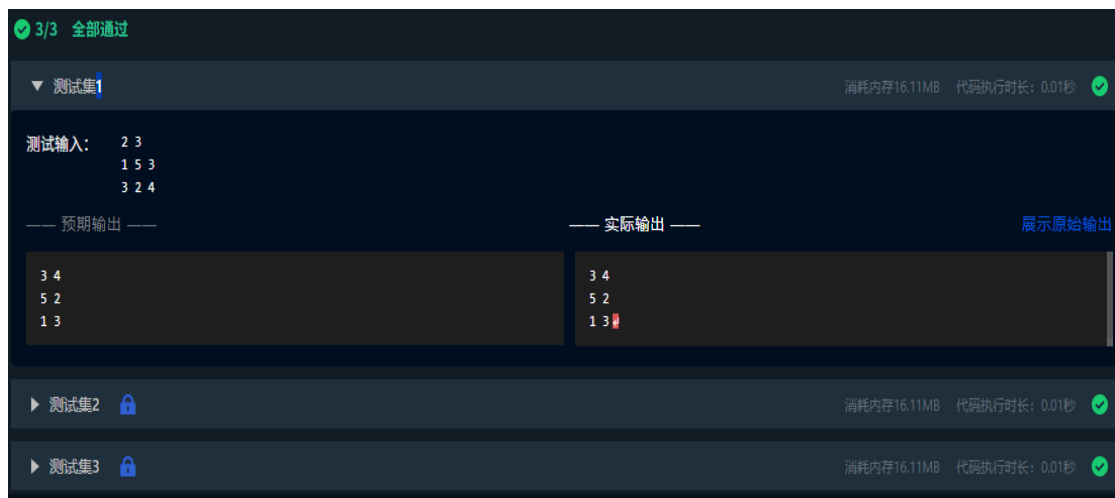


图 1-3 程序设计题 1.3.3 的流程图

我的源代码如下：

```
#include<stdio.h>

int main(){
    int n,m;
    scanf("%d %d",&n,&m);
    int mat[100][100],rotated[100][100];
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("%d",&mat[i][j]);
        }
    }
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            rotated[m-1-j][i]=mat[i][j];
        }
    }
    for(int i=0;i<m;i++){
        printf("%d",rotated[i][0]);
        for(int j=1;j<n;j++){
            printf(" %d",rotated[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```



4. 命令行实现对 N 个整数排序

本关要求通过命令行实现对 N 个整数排序。

我的思路是：先定义升序（sort_up）和降序（sort_down）两个排序函数，均采用冒泡排序实现。主函数通过命令行参数（argc/argv）获取输入，第一个参数为待排序整数个数 n，第二个参数“-d”控制降序。先校验参数合法性，再读取 n 个整数存入数组，根据参数调用对应排序函数，最后遍历输出排序后的数组，完成命令行下的整数排序。

我的源代码如下：

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

void sort_up(int a[],int n){
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-1-i;j++){
            if(a[j]>a[j+1]){
                t=a[j];a[j]=a[j+1];a[j+1]=t;
            }
        }
}

void sort_down(int a[],int n){
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-1-i;j++){
            if(a[j]<a[j+1]){
                t=a[j];a[j]=a[j+1];a[j+1]=t;
            }
        }
}

int main(int argc,char*argv[]){
    if(argc<2||argc>3){return 1;}
    int n=atoi(argv[1]);
    if(n<=0){return 1;}
    int a[100];
    for(int i=0;i<n;i++)scanf("%d",&a[i]);
    if(argc==3&&strcmp(argv[2],"-d")==0){
        sort_down(a,n);
    }else{
        sort_up(a,n);
    }
    for(int i=0;i<n;i++)printf("%d ",a[i]);
    printf("\n");
    return 0;
}
```



5. 删除子串

本关要求编写函数 `delSubstr(str, substr)`，删除字符串 `str` 中出现的所有子串 `substr`。如果成功删除子串，则返回 1，若 `str` 中不包含子串 `substr`，则返回 0。函数中的任何地方都不使用下标引用，在 `main` 函数中输出删除子串后的结果字符串。

我的思路是：先调用 `trimSpace` 函数，用指针把输入的原字符串 `str`、要删的子串 `substr` 开头和结尾的空格都去掉；然后调 `delSubstr` 函数，用指针扫描 `str` 里的字符，一段段和 `substr` 配对，配对成功就跳过这个子串，没配对上的话就把当前这个字符存到临时数组 `temp` 里，等扫描完了再把 `temp` 里的内容放回 `str`；最后把处理好的 `str` 和 `delSubstr` 返回的标记打印出来。

我的流程图如下图 1-4：

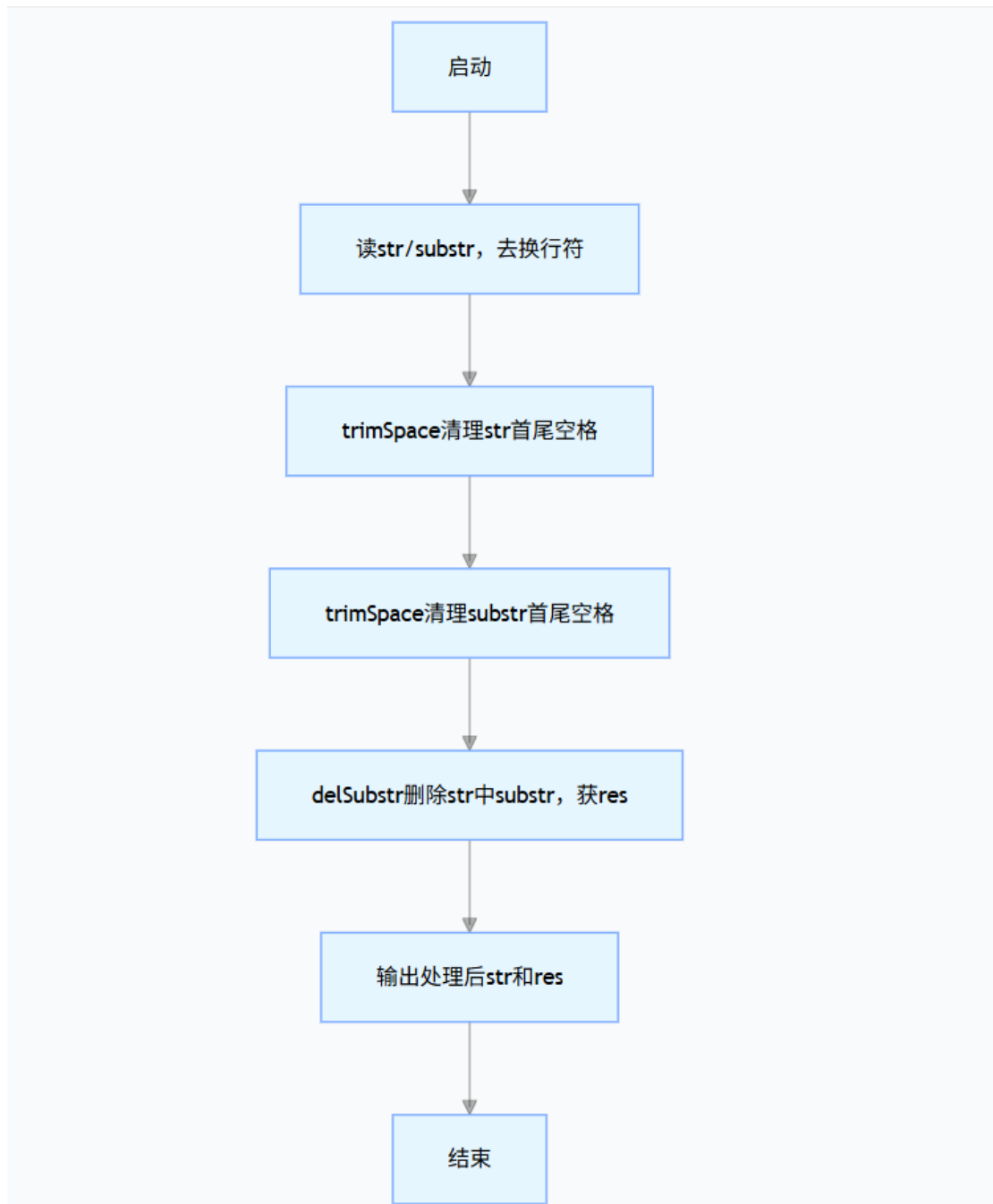


图 1-4 程序设计题 1.3.5 的流程图

我的源代码如下：

```

#include<stdio.h>
#include<string.h>

void trimSpace(char *str) {
    char *start = str, *end = str, *p = str;
    while (*p == ' ') p++;
    start = p;
    while (*p) p++;
    end = p - 1;
    while (end >= start && *end == ' ') end--;
    *(end + 1) = '\0';
    p = str;
    while (*start) *p++ = *start++;
    *p = '\0';
}

```

```

int delSubstr(char *str, char *substr) {
    char temp[1024] = {0};
    char *p = str, *t = temp, *s;
    int sub_len = strlen(substr);
    int flag = 0;
    if (sub_len == 0) return 0;
    while (*p) {
        s = substr;
        char *q = p;
        while (*s && *q == *s) {q++; s++;}
        if (!*s) {
            flag = 1;
            p += sub_len;
        } else {
            *t++ = *p++;
        }
    }
}

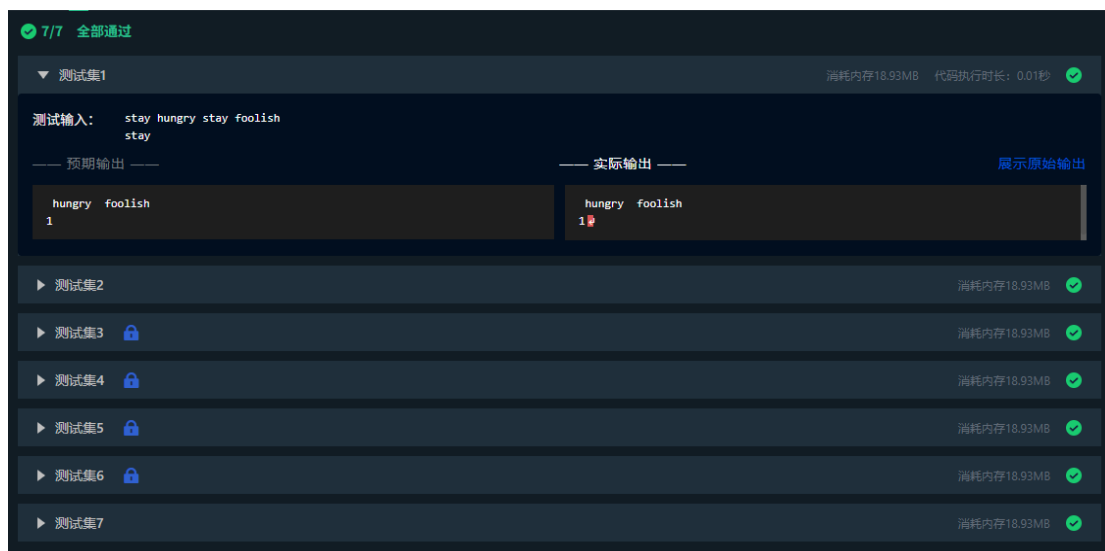
```

```

    strcpy(str, temp);
    return flag;
}

int main() {
    char str[1024], substr[100];
    fgets(str, 1024, stdin);
    str[strcspn(str, "\n")] = '\0';
    fgets(substr, 100, stdin);
    substr[strcspn(substr, "\n")] = '\0';
    trimSpace(str);
    trimSpace(substr);
    int res = delSubstr(str, substr);
    printf("%s\n%d\n", str, res);
    return 0;
}

```



6. 非负整数积

本关的要求是求两个不超过 200 位的非负整数积。

我的思路是：要算两个最多 200 位的数相乘，普通整数类型没办法存下这么大的数字，没法直接算，所以使用竖式乘法的方式来实现。先定义字符数组把输入的两个大数字字符串存下来，再转成整型数组方便逐位计算。然后从两个数的个位开始，一位一位相乘，每乘出来的结果先存到结果数组对应的位置，同时把超

过 10 的数进位到高位；等所有位都乘完后，把结果数组里开头没用的零去掉，最后从高位到低位把结果打印出来。

我的流程图如下图 1-3：

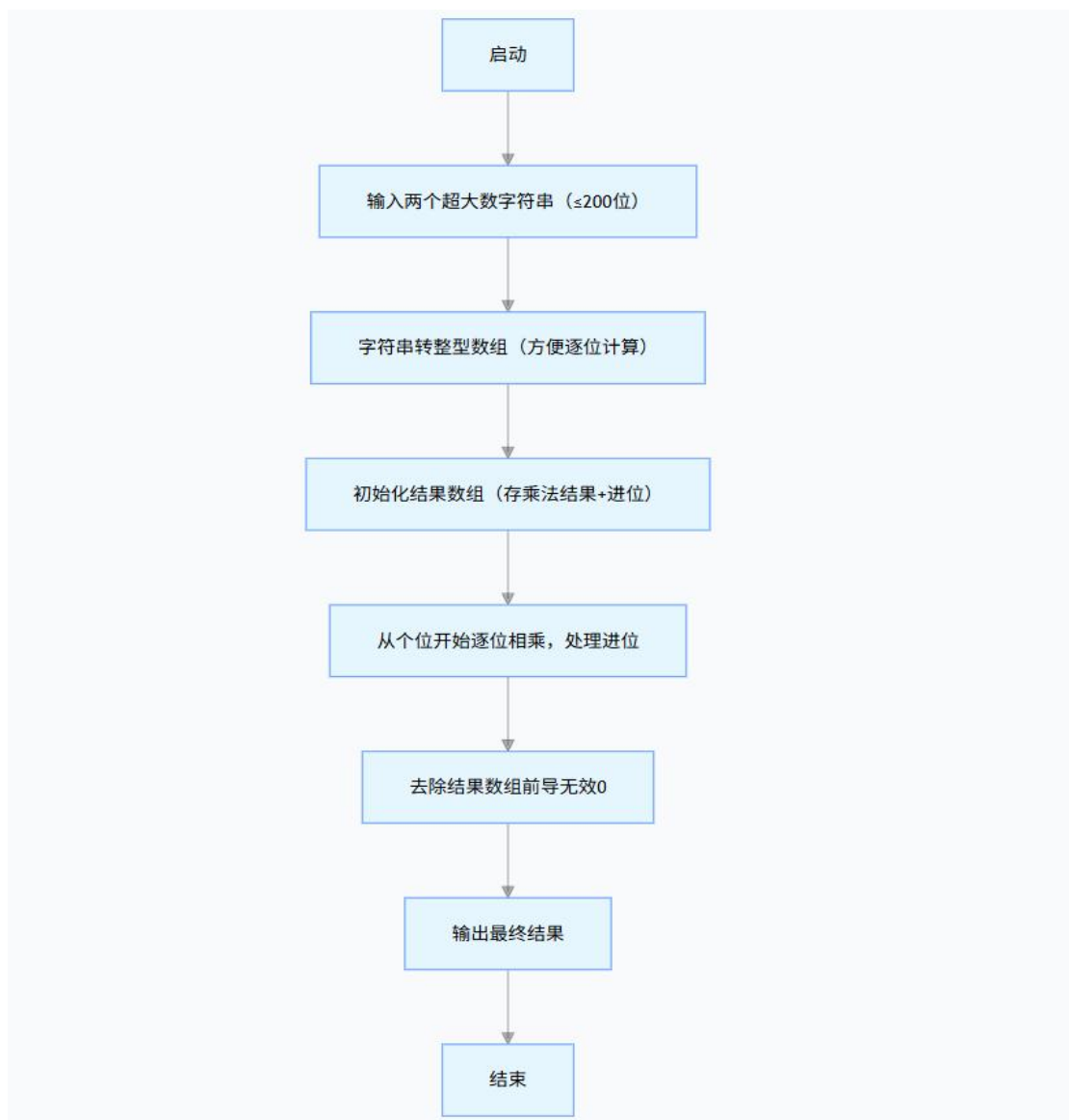


图 1-5 程序设计题 1.3.6 的流程图

我的源代码如下：

```

#include<stdio.h>
#include<string.h>

#define MAX 200
#define RES 400

int main(){
    char s1[MAX+1],s2[MAX+1];
    int n1[MAX]={0},n2[MAX]={0},res[RES]={0};
    int l1,l2,i,j,p,c;

    scanf("%s%s",s1,s2);
    l1=strlen(s1);
    l2=strlen(s2);

    for(i=0;i<l1;i++)n1[i]=s1[i]-'0';
    for(i=0;i<l2;i++)n2[i]=s2[i]-'0';

    for(i=l1-1;i>=0;i--){
        c=0;
        for(j=l2-1;j>=0;j--){
            res[(l1-1-i)+(l2-1-j)]+=n1[i]*n2[j]+c;
            c=res[(l1-1-i)+(l2-1-j)]/10;
            res[(l1-1-i)+(l2-1-j)]%=10;
        }
        if(c>0)res[(l1-1-i)+l2]=c;
    }

    p=l1+l2-1;
    while(p>0&&res[p]==0)p--;

    for(i=p;i>=0;i--)printf("%d",res[i]);
    printf("\n");

    return 0;
}

```




7. 函数调度

本关要求编写 8 个任务函数，一个 scheduler 调度函数和一个 execute 执行函数，通过函数指针建立转移表，通过函数调度实现对应功能。

我的思路是：先用函数指针数组建立转移表，把 0-7 的数字编号和 task0 到 task7 的函数配对。编写 scheduler 调度函数负责检验输入的字符串，逐个识别字符，如果不是数字直接跳过，是数字则转成任务编号。execute 执行函数负责检验编号是否在 0-7 范围内，符合就通过转移表调用对应任务函数，不符合则提示超出范围。

我的源代码如下：

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void task0() { puts("task0 is called!"); }
void task1() { puts("task1 is called!"); }
void task2() { puts("task2 is called!"); }
void task3() { puts("task3 is called!"); }
void task4() { puts("task4 is called!"); }
void task5() { puts("task5 is called!"); }
void task6() { puts("task6 is called!"); }
void task7() { puts("task7 is called!"); }
```

```

void execute(int task_id, void (*fun_table[])(void), int max_id) {
    if (task_id >= 0 && task_id <= max_id) {
        fun_table[task_id]();
    } else {
        printf("超出范围\n");
    }
}

void scheduler(const char *input_str, void (*fun_table[])(void), int max_id) {
    if (input_str == NULL || strlen(input_str) == 0) {
        printf("输入为空\n");
        return;
    }

    for (int i = 0; input_str[i] != '\0'; i++) {
        char curr_char = input_str[i];
        if (!isdigit(curr_char)) {
            printf("字符 '%c' 非数字\n", curr_char);
            continue;
        }
        int task_id = curr_char - '0';
        execute(task_id, fun_table, max_id);
    }
}

int main() {
    void (*task_funs[])(void) = {task0, task1, task2, task3, task4, task5, task6, task7};
    const int MAX_TASK_ID = sizeof(task_funs) / sizeof(task_funs[0]) - 1;

    char input_str[20];
    scanf("%19s", input_str);

    scheduler(input_str, task_funs, MAX_TASK_ID);

    return 0;
}

```

3/3 全部通过

测试集1

消耗内存14MB 代码执行时长: 0.01秒

测试输入: 13607122

—— 预期输出 ——

```

task1 is called!
task3 is called!
task6 is called!
task0 is called!
task7 is called!
task1 is called!
task2 is called!
task2 is called!

```

—— 实际输出 ——

```

task1 is called!
task3 is called!
task6 is called!
task0 is called!
task7 is called!
task1 is called!
task2 is called!
task2 is called!

```

[展示原始输出](#)

测试集2

消耗内存14MB 代码执行时长: 0.01秒

测试集3

消耗内存14MB

1.4 小结

在 C 语言里，指针确实是个又强又灵活的工具，但刚开始学的时候，我只觉得它很抽象。直到做完这次实验，我才真正明白，它的核心就是存另一个变量的内存地址，能直接操作内存，这也是为啥用指针写的代码效率高，而且动态内存管理、链表这种复杂数据结构，都离不开它。

这次实验让我对指针的认知，终于从零散的知识点串成了体系。基础的地址操作、指针偏移这些语法，我练得更扎实了；更重要的是，不再是只会背语法，而是能在实际编程里用起来了。比如函数传参，之前总搞不懂为啥值传递改不了外部变量，后来用指针传地址，看着代码里 `*p` 直接修改外部变量的值，一下子就通了。还有动态内存分配，用 `malloc` 申请空间后，得用指针接收地址，用完再用 `free` 释放。并且处理字符串的时候，用指针遍历也很方便。

当然，实验也暴露了我的不足。比如多维指针，尤其是二级指针和二维数组结合的时候，我写函数参数的时候总写错；还有函数指针、指针数组这种复杂类型声明，

这段学习过程真的让我感触很深：掌握指针不只是为了写出效率高的代码，更重要的是逼着自己养成严谨的编程思维。以前写代码只想着能跑就行，现在用指针的时候，每一步都要想清楚指针指向的是哪块内存、地址对不对、有没有释放，不然稍微错一点就是段错误。接下来我会继续学习指针的更深度的内容，希望加深对它的理解。

2 实验 7 结构与联合

2.1 表达式求值的程序验证

我的分析过程是，首先明确运算符的优先级顺序，比如：括号的优先级最高，其次是结构体指针访问符号 $\rightarrow$ ，再往后是自增符号 $++$ 和间接访问符号 $*$ 。然后再根据优先顺序一步步求解。具体分析如下：

当 $n=1$ 时，执行 $(++p)\rightarrow x$ ：因为括号优先级高，先执行 $++p$ 操作，使 p 从指向 $a[0]$ 变为指向 $a[1]$ ；再通过 $\rightarrow x$ 访问 $a[1]$ 的成员 x ，最终输出结果为 100。

当 $n=2$ 时，先执行 $p++$ ，再执行 $\text{printf}(\text{"\%c"}, p\rightarrow c)$ ：先执行 $p++$ 操作，执行完该语句后 p 指向 $a[1]$ ，随后 $p\rightarrow c$ 访问 $a[1]$ 的成员 c ，最后输出结果为 B。

当 $n=3$ ，先执行 $*p++\rightarrow t$ ，再执行 $\text{printf}(\text{"\%c"}, *p\rightarrow t)$ ； $\rightarrow$ 优先级高于 $++$ ， $p++$ 使 p 从指向 $a[0]$ 变为指向 $a[1]$ ，随后 $p\rightarrow t$ 访问 $a[1]$ 的 t 指向的首个字符，最后输出结果为 x。

当 $n=4$ 时，执行 $(++p)\rightarrow t$ ：因为括号优先级高，先执行 $++p$ 操作，使 p 从指向 $a[0]$ 变为指向 $a[1]$ ；再通过 $\rightarrow t$ 访问 $a[1]$ 的 t 并间接访问，最后输出结果为 x。

当 $n=5$ 时，执行 $*++p\rightarrow t$ ：因为 $\rightarrow$ 优先级高于 $++$ ，先执行 $p\rightarrow t$ 获取 $a[0]$ 中的 t 的地址，再执行 $++$ 操作使地址指向 V；随后间接访问该地址，最后输出结果为 V。

当 $n=6$ 时，执行 $++p\rightarrow t$ ：因为 $\rightarrow$ 优先级高于 $++$ ，先执行 $p\rightarrow t$ 获取 $a[0]$ 的 t 的地址，再间接访问取出字符 U；随后 $++$ 使字符变为 V，最后输出结果为 V。

具体代码如下：

```
#include<stdio.h>
char u[] = "UVWXYZ",v[]="xyz";
struct T{
    int x;
    char c;
    char *t;

} a[] = {
    {11, 'A', u},
    {100, 'B', v}
} ,*p=a;
```

```

int main()
{
    int n;
    scanf("%d",&n);
    switch(n)
    {
        case 1 :
            printf("%d",(++p)->x);
            break;
        case 2:
            p++;
            printf("%c",p->c);
            break;
        case 3:
            *p++->t;
            printf("%c",*p->t);
            break;
        case 4:
            printf("%c",*(++p)->t);
            break;
        case 5:
            printf("%c",*++p->t);
            break;
        case 6:
            printf("%c",++*p->t);
            break;
        default :
            break;
    }
    return 0;
}

```

下图为验证结果：

1	2	3	4	5	6
100	B	x	x	V	V

2.2 源程序修改替换

1. 源程序修改替换（一）

我把原来的固定数组改成控制台动态输入，在 main 函数里加了链表，并且 create_list 函数里补充了空输入的处理，最后把尾结点的 next 赋值成 NULL，避免尾结点指针指向随机内存。

我修改替换的代码如下：

```
#include <stdio.h>
#include <stdlib.h>

struct s_list {
    int data;
    struct s_list *next;
};
```

```
void create_list(struct s_list **headp, int *p);
```

```
int main(void) {
    struct s_list *head = NULL, *p;
    int s[100];
    int i = 0, num;

    while (1) {
        scanf("%d", &num);
        s[i++] = num;
        if (num == 0) {
            break;
        }
    }

    create_list(&head, s);

    p = head;
    while (p) {
        printf("%d\t", p->data);
        p = p->next;
    }
    printf("\n");

    return 0;
}
```

```

void create_list(struct s_list **headp, int *p) {
    struct s_list *loc_head = NULL, *tail;

    if (p[0] == 0) {
        *headp = NULL;
        return;
    }

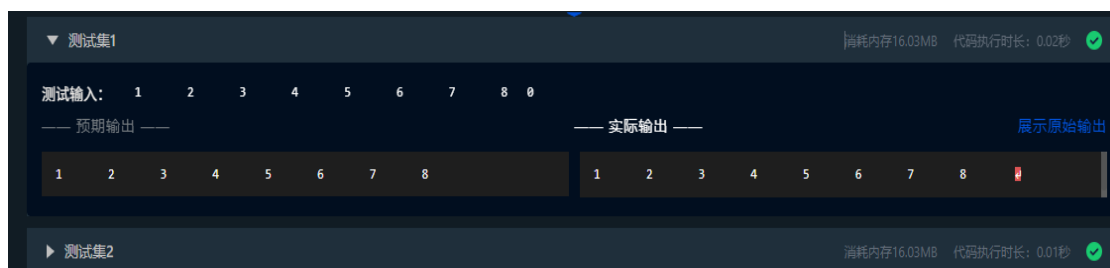
    loc_head = (struct s_list *)malloc(sizeof(struct s_list));
    loc_head->data = *p++;
    tail = loc_head;

    while (*p != 0) {
        tail->next = (struct s_list *)malloc(sizeof(struct s_list));
        tail = tail->next;
        tail->data = *p++;
    }
    tail->next = NULL;

    *headp = loc_head;
}

```

下图是修改后的代码运行结果：



2. 源程序修改替换（二）

本关要求修改替换 create_list 函数，将其建成一个后进先出的链表。我将该函数的链表创建逻辑从尾插法改成了头插法，让新结点始终指向当前头结点并成为新的头结点。

我修改替换的代码如下：

```

#include <stdio.h>
#include <stdlib.h>

struct s_list {
    int data;
    struct s_list *next;
};

void create_list(struct s_list **headp, int *p);

```

```

int main(void) {
    struct s_list *head = NULL, *p;
    int s[100];
    int i = 0, num;

    while (1) {
        scanf("%d", &num);
        s[i++] = num;
        if (num == 0) {
            break;
        }
    }

    create_list(&head, s);

    p = head;
    while (p) {
        printf("%d\t", p->data);
        p = p->next;
    }
    printf("\n");

    return 0;
}

```

```

void create_list(struct s_list **headp, int *p) {
    struct s_list *new_node;

    if (p[0] == 0) {
        *headp = NULL;
        return;
    }

    while (*p != 0) {
        new_node = (struct s_list *)malloc(sizeof(struct s_list));
        new_node->data = *p++;
        new_node->next = *headp;
        *headp = new_node;
    }
}

```

下图是修改后的代码运行结果：



2.3 程序设计

1. 设计字段结构

本关要求设计一个字段结构 `struct bits`, 它将一个 8 位无符号字节从最低位到最高位声明为 8 个字段, 依次为 `bit0, bit1, ..., bit7` 且 `bit0` 优先级最高。同时设计 8 个函数, 第 `i` 个函数以 `biti (i=0, 1, ..., 7)` 为参数, 并且在函数体内输出 `biti` 的值。将 8 个函数的名字存入一个函数指针数组 `p_fun`。如果 `bit0` 为 1, 调用 `p_fun[0]` 指向的函数。如果 `struct bits` 中有多位为 1, 则根据优先级从高到低顺序依次调用函数指针数组 `p_fun` 中相应元素指向的函数。

我首先定义了一个结构体, 里面包含 8 个 `int` 成员, 用来存无符号整数的 8 个二进制位; 又定义 8 个标识性函数, 还有指向这些函数的指针数组。接下来读取用户输入的无符号整数, 用位运算把这个数 8 个二进制位拆出来, 挨个存到结构体对应的成员里; 最后遍历结构体的 8 个成员, 只要某个成员值是 1, 就通过函数指针数组调用对应的函数。

我的流程图如下图 2-1:



图 2-1 程序设计题 2.3.1 的流程图

我的源代码如下：

```
#include<stdio.h>
struct bits
{
    int bit0;
    int bit1;
    int bit2;
    int bit3;
    int bit4;
    int bit5;
    int bit6;
    int bit7;
} bits;
void f0(int b)
{
    printf("the function %d is called!\n",b);
}
void f1(int b){
    printf("the function %d is called!\n",b);
}
void f2(int b){
    printf("the function %d is called!\n",b);
}
void f3(int b){
    printf("the function %d is called!\n",b);
}
```

```

void f4(int b){
    printf("the function %d is called!\n",b);
}
void f5(int b){
    printf("the function %d is called!\n",b);
}
void f6(int b){
    printf("the function %d is called!\n",b);
}
void f7(int b){
    printf("the function %d is called!\n",b);
}
int main()
{
    void (*p[])(int)={f0,f1,f2,f3,f4,f5,f6,f7};
    int * a= &bits.bit0;
    int * b= &bits.bit0;
    unsigned int n;
    scanf("%u",&n);
    for(int k=0;k<8;k++)
    {
        *a=((n>>k) & 1);
        a++;
    }
    for(int k=0;k<8;k++)
    {
        if(*b) p[k](k);
        b++;
    }

    return 0;
}

```

2. 班级成绩单

本关要求用单向链表建立一张班级成绩单，包括每个学生的学号、姓名、英语、高等数学、普通物理、C 语言程序设计 4 门课程的成绩。并实现菜单中的功能。

我首先定义了一个结构体，里面存由学生的学号、姓名，还有四门科目的成绩，另外加个链表指针；然后用全局头指针来管整个链表。程序运行后会循环让用户输入功能指令，输不同的指令就调对应的函数。

相关函数及其功能：

input()：批量录入 n 个学生信息，用尾插法将学生添加到单链表尾部。

print2()：遍历链表，逐个打印学生的学号、姓名及四门科目成绩。

alter()：接收学生学号、科目编号、新成绩，找到对应学生并修改指定科目成绩。

avg()：遍历链表，计算每位学生四门科目平均分，输出学号、姓名及平均

分。

sum_avg(): 遍历链表, 计算每位学生四门科目总分和平均分, 输出学号、姓名、总分及平均分。

我的源代码如下:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct Stu{
    char id[20];
    char name[20];
    int eng;
    int math;
    int phy;
    int c;
    struct Stu *next;
};
struct Stu *head=NULL;
void input(){
    int n;
    scanf("%d",&n);
    struct Stu *tail=head;
    if(tail!=NULL){
        while(tail->next!=NULL){
            tail=tail->next;
        }
    }
    for(int i=0;i<n;i++){
        struct Stu *new=(struct Stu *)malloc(sizeof(struct Stu));
        scanf("%s%s%d%d%d",new->id,new->name,&new->eng,&new->math,&new->phy,&new->c);
        new->next=NULL;
        if(head==NULL){
            head=new;
            tail=new;
        }else{
            tail->next=new;
            tail=new;
        }
    }
}
void print2(){
    struct Stu *p=head;
    while(p!=NULL){
        printf("%s %s %d %d %d %d\n",p->id,p->name,p->eng,p->math,p->phy,p->c);
        p=p->next;
    }
}
```

```

void alter(){
    char tid[20];
    int num,score;
    scanf("%s%d%d",tid,&num,&score);
    struct Stu *p=head;
    while(strcmp(p->id,tid)!=0){
        p=p->next;
    }
    if(num==1)p->eng=score;
    if(num==2)p->math=score;
    if(num==3)p->phy=score;
    if(num==4)p->c=score;
}
void avg(){
    struct Stu *p=head;
    while(p!=NULL){
        float a=(p->eng+p->math+p->phy+p->c)/4.0;
        printf("%s %s %.2f\n",p->id,p->name,a);
        p=p->next;
    }
}

```

```

void sum_avg(){
    struct Stu *p=head;
    while(p!=NULL){
        int sum=p->eng+p->math+p->phy+p->c;
        float a=sum/4.0;
        printf("%s %s %d %.2f\n",p->id,p->name,sum,a);
        p=p->next;
    }
}

```

```

int main(){
    int cmd;
    while(1){
        scanf("%d",&cmd);
        if(cmd==0)break;
        if(cmd==1)input();
        if(cmd==2)print2();
        if(cmd==3)alter();
        if(cmd==4)avg();
        if(cmd==5)sum_avg();
    }
    return 0;
}

```

3. 成绩排序（一）

本关要求对程序设计的第二题增加按照平均成绩进行升序排序的函数，写出用交换结点数据域的方法升序排序的函数。

我在第 2 题基础上，增加 sort_by_avg() 函数通过冒泡排序对链表按平均分升序排序，并在 print2(), avg(), sum_avg() 函数执行输出前调用 sort_by_avg() 函数，使打印和统计结果均按平均分升序展示。

我的源代码如下：

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct Stu{
    char id[20];
    char name[20];
    int eng;
    int math;
    int phy;
    int c;
    struct Stu *next;
};
struct Stu *head=NULL;

void sort_by_avg(){
    if(head==NULL || head->next==NULL)return;
    struct Stu *p,*q;
    int flag;

    do{
        flag=0;
        p=head;
        while(p->next!=NULL){
            q=p->next;

            float avg_p=(p->eng+p->math+p->phy+p->c)/4.0;
            float avg_q=(q->eng+q->math+q->phy+q->c)/4.0;
```

```

        if(avg_p>avg_q){

            char tmp_id[20];
            strcpy(tmp_id,p->id);
            strcpy(p->id,q->id);
            strcpy(q->id,tmp_id);

            char tmp_name[20];
            strcpy(tmp_name,p->name);
            strcpy(p->name,q->name);
            strcpy(q->name,tmp_name);

            int tmp=0;
            tmp=p->eng;p->eng=q->eng;q->eng=tmp;
            tmp=p->math;p->math=q->math;q->math=tmp;
            tmp=p->phy;p->phy=q->phy;q->phy=tmp;
            tmp=p->c;p->c=q->c;q->c=tmp;
            flag=1;
        }
        p=p->next;
    }
}while(flag);
}

```

```

void input(){
    int n;
    scanf("%d",&n);
    struct Stu *tail=head;
    if(tail!=NULL){
        while(tail->next!=NULL){
            tail=tail->next;
        }
    }
    for(int i=0;i<n;i++){
        struct Stu *new=(struct Stu *)malloc(sizeof(struct Stu));
        scanf("%s%s%d%d%d",new->id,new->name,&new->eng,&new->math,&new->phy,&new->c);
        new->next=NULL;
        if(head==NULL){
            head=new;
            tail=new;
        }else{
            tail->next=new;
            tail=new;
        }
    }
}

```

```

void print2(){
    sort_by_avg();
    struct Stu *p=head;
    while(p!=NULL){
        printf("%s %s %d %d %d %d\n",p->id,p->name,p->eng,p->math,p->phy,p->c);
        p=p->next;
    }
}

```

```

void alter(){
    char tid[20];
    int num,score;
    scanf("%s%d%d",tid,&num,&score);
    struct Stu *p=head;
    while(strcmp(p->id,tid)!=0){
        p=p->next;
    }
    if(num==1)p->eng=score;
    if(num==2)p->math=score;
    if(num==3)p->phy=score;
    if(num==4)p->c=score;
}

```

```

void avg(){
    sort_by_avg();
    struct Stu *p=head;
    while(p!=NULL){
        float a=(p->eng+p->math+p->phy+p->c)/4.0;
        printf("%s %s %.2f\n",p->id,p->name,a);
        p=p->next;
    }
}

```



```

void sum_avg(){
    sort_by_avg();
    struct Stu *p=head;
    while(p!=NULL){
        int sum=p->eng+p->math+p->phy+p->c;
        float a=sum/4.0;
        printf("%s %s %d %.2f\n",p->id,p->name,sum,a);
        p=p->next;
    }
}

int main(){
    int cmd;
    while(1){
        scanf("%d",&cmd);
        if(cmd==0)break;
        if(cmd==1)input();
        if(cmd==2)print2();
        if(cmd==3)alter();
        if(cmd==4)avg();
        if(cmd==5)sum_avg();
    }
    return 0;
}

```

4. 回文字符串

本关要求设计程序判断输入的任意长度的字符串是否为回文字符串

我是这么想的：由于要求字符串长度任意，所以用单链表存储字符串。于是我先写了 createLinkList 函数，传入字符串和链表头指针地址，用尾插法把每个字符做成节点放到链表尾，把字符串转成字符单链表；然后 judgePalindrome 函数先靠快慢指针定位链表中点，反转中点后的链表，再用两个指针分别从表头和反转后的后半段表头比对字符，全一致就是回文(true)，不然就不是(false)。

我的流程图如下图 2-2：

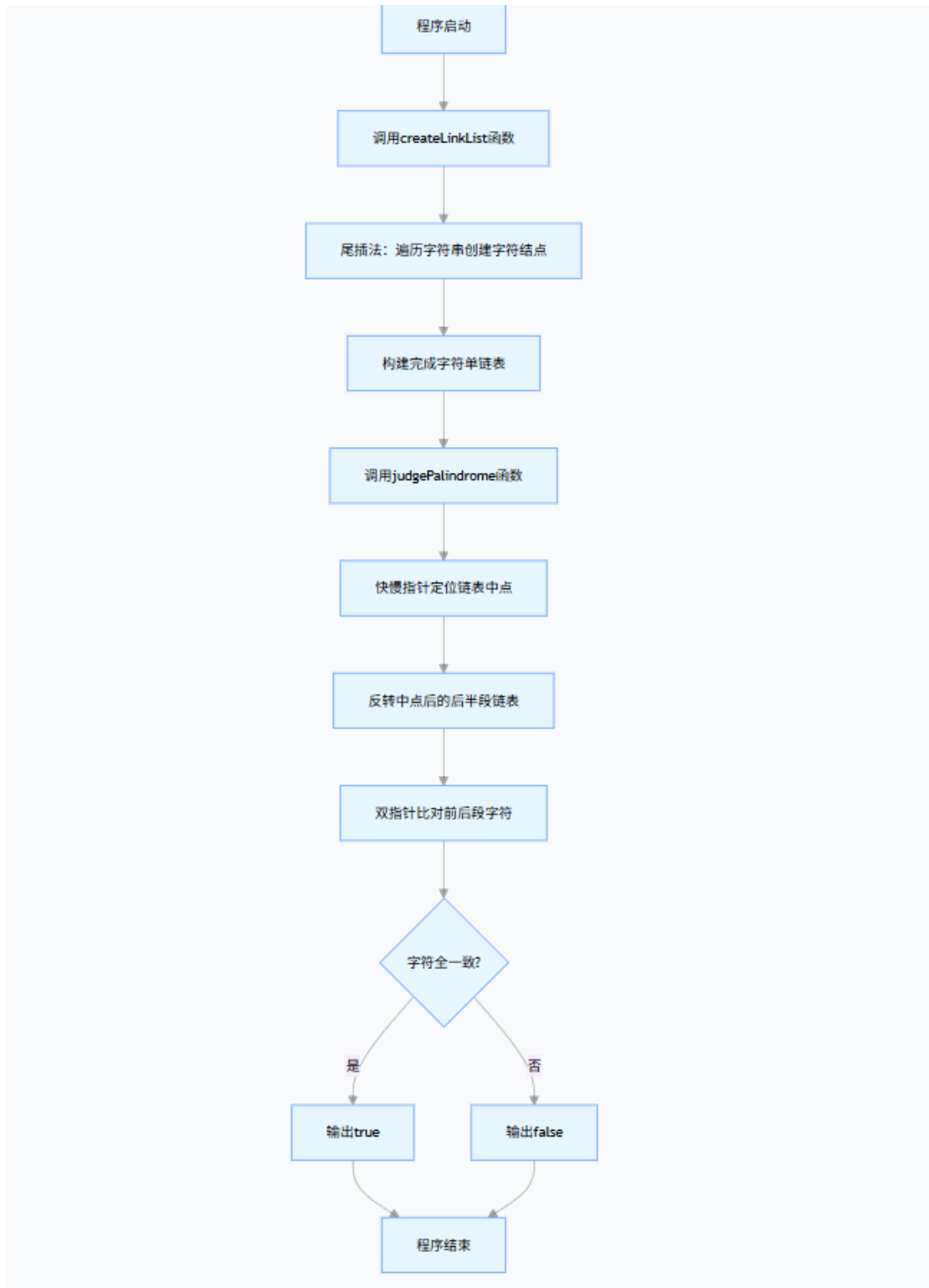


图 2-2 程序设计题 2.3.4 的流程图

我的源代码如下：

/*测试程序定义了如下结点类型

```
typedef struct c_node{
    char data; struct c_node *next;
} C_NODE;
```

*****/

```
void createLinkList(C_NODE **headp, char s[])
```

```
{
    /***** BEGIN *****/
    *headp=NULL;
    C_NODE *tail=NULL;
    int len=strlen(s);
    for(int i=0;i<len;i++){
        C_NODE *new_node=(C_NODE *)malloc(sizeof(C_NODE));
        new_node->data=s[i];
        new_node->next=NULL;
        if(*headp==NULL){
            *headp=new_node;
            tail=new_node;
        }else{
            tail->next=new_node;
            tail=new_node;
        }
    }
    /***** BEGIN *****/
}
```

```
void judgePalindrome(C_NODE *head)
```

```
{
    /***** BEGIN *****/
    if(head==NULL){
        printf("false\n");
        return;
    }
    C_NODE *slow=head,*fast=head;
    while(fast!=NULL&&fast->next!=NULL){
        slow=slow->next;
        fast=fast->next->next;
    }
    C_NODE *pre=NULL,*cur=slow,*next;
    while(cur!=NULL){
        next=cur->next;
        cur->next=pre;
        pre=cur;
        cur=next;
    }
}
```

```

C_NODE *p1=head,*p2=pre;
int is_pal=1;
while(p2!=NULL){
    if(p1->data!=p2->data){
        is_pal=0;
        break;
    }
    p1=p1->next;
    p2=p2->next;
}
is_pal?printf("true\n"):printf("false\n");
/***** BEGIN *****/
}

```

5. 成绩排序（二）

本关要求对第 3 题进一步写出用交换结点指针域的方法升序排序的函数。

我对第 3 关的代码进行了修改，把原来的“交换节点数据”改成了“交换链表的指针域”。原来的思路是遍历链表，要是前一个节点的平均分比后一个高，就直接把两个节点里的学号、姓名、四门成绩这些数据全互换，但是这样不仅数据多而且操作起来费时间。于是我额外添加了一个固定的节点，从而不用再单独处理头节点，排序时也不需要再动节点里的数据，只通过“pre->next=q、p->nex=q->next、q->next=p”这三步把节点位置换过来。同时排序方法依然使用冒泡排序进行升序排序。

我的源代码如下：

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct Stu{
    char id[20];
    char name[20];
    int eng;
    int math;
    int phy;
    int c;
    struct Stu *next;
};
struct Stu *head=NULL;

// 交换指针域实现升序排序（不再交换节点数据）
void sort_by_avg(){
    if(head==NULL||head->next==NULL)return;

```

```

struct Stu dummy;
dummy.next = head;
struct Stu *pre, *p, *q;
int flag;

do{
    flag=0;
    pre = &dummy;
    p = pre->next;
    while(p!=NULL && p->next!=NULL){
        q = p->next;

        float avg_p=(p->eng+p->math+p->phy+p->c)/4.0;
        float avg_q=(q->eng+q->math+q->phy+q->c)/4.0;

        if(avg_p>avg_q){
            pre->next = q;    // 前驱指向后节点
            p->next = q->next; // 前节点指向后节点的下一个
            q->next = p;      // 后节点指向前节点
            flag=1;          // 标记有交换

            // 交换后重置p (pre不变, p指向新的pre->next)
            p = pre->next;
        }
    }
}while(flag);

head = dummy.next; // 更新头节点为排序后的链表头
}

void input(){
    int n;
    scanf("%d",&n);
    struct Stu *tail=head;
    if(tail!=NULL){
        while(tail->next!=NULL){
            tail=tail->next;
        }
    }
    for(int i=0;i<n;i++){
        struct Stu *new=(struct Stu *)malloc(sizeof(struct Stu));
        scanf("%s%s%d%d%d%d",new->id,new->name,&new->eng,&new->math,&new->phy,&new->c);
        new->next=NULL;
        if(head==NULL){
            head=new;
            tail=new;
        }else{

```

```

        }else{
            tail->next=new;
            tail=new;
        }
    }
}

void print2(){
    sort_by_avg();
    struct Stu *p=head;
    while(p!=NULL){
        printf("%s %s %d %d %d %d\n",p->id,p->name,p->eng,p->math,p->phy,p->c);
        p=p->next;
    }
}

```

```

void alter(){
    char tid[20];
    int num,score;
    scanf("%s%d%d",tid,&num,&score);
    struct Stu *p=head;
    while(strcmp(p->id,tid)!=0){
        p=p->next;
    }
    if(num==1)p->eng=score;
    if(num==2)p->math=score;
    if(num==3)p->phy=score;
    if(num==4)p->c=score;
}

```

```

void avg(){
    sort_by_avg();
    struct Stu *p=head;
    while(p!=NULL){
        float a=(p->eng+p->math+p->phy+p->c)/4.0;
        printf("%s %s %.2f\n",p->id,p->name,a);
        p=p->next;
    }
}

```

```

void sum_avg(){
    sort_by_avg();
    struct Stu *p=head;
    while(p!=NULL){
        int sum=p->eng+p->math+p->phy+p->c;
        float a=sum/4.0;
        printf("%s %s %d %.2f\n",p->id,p->name,sum,a);
        p=p->next;
    }
}

```

```
int main(){
    int cmd;
    while(1){
        scanf("%d",&cmd);
        if(cmd==0)break;
        if(cmd==1)input();
        if(cmd==2)print2();
        if(cmd==3)alter();
        if(cmd==4)avg();
        if(cmd==5)sum_avg();
    }
    return 0;
}
```

6. 逆波兰表达式

本关要求利用值栈对逆波兰表达式进行求值。逆波兰表达式从键盘输入，其中的运算符仅包含加、减、乘、除 4 种运算，表达式中的数都是十进制数，用换行符结束输入。

我的思路是用单链表模拟栈来计算逆波兰表达式：先定义栈节点结构体 StackNode，里面存储浮点型数值 value 和指向下一节点的指针 next，并用全局指针 top 标记栈顶；接着用 push 函数，通过新建节点存入要压栈的数值，让新节点的 next 指向当前栈顶并更新 top 为新节点实现入栈，写 pop 函数，取出栈顶节点的数值后将 top 移至下一个节点，释放原栈顶节点实现出栈；核心的 evaluate_rpn 函数会用 scanf 循环读取输入的每个 token。先判断 token 是否为数字，若是则用 atof 转换为浮点型入栈，若为运算符则先弹出两个数，通过 switch 判断运算符类型完成计算，如果是除法需先判断除数 b 是否为 0（为 0 则报错退出）且强制整数除法，计算结果压回栈中，遇到无效运算符也直接报错退出；所有 token 处理完毕后，弹出栈中最终结果，清空剩余栈节点避免内存泄漏并返回结果；main 函数接收结果后，判断结果是否为整数，若是则按整型输出，否则按浮点型输出。

我的流程图如下图 2-3：

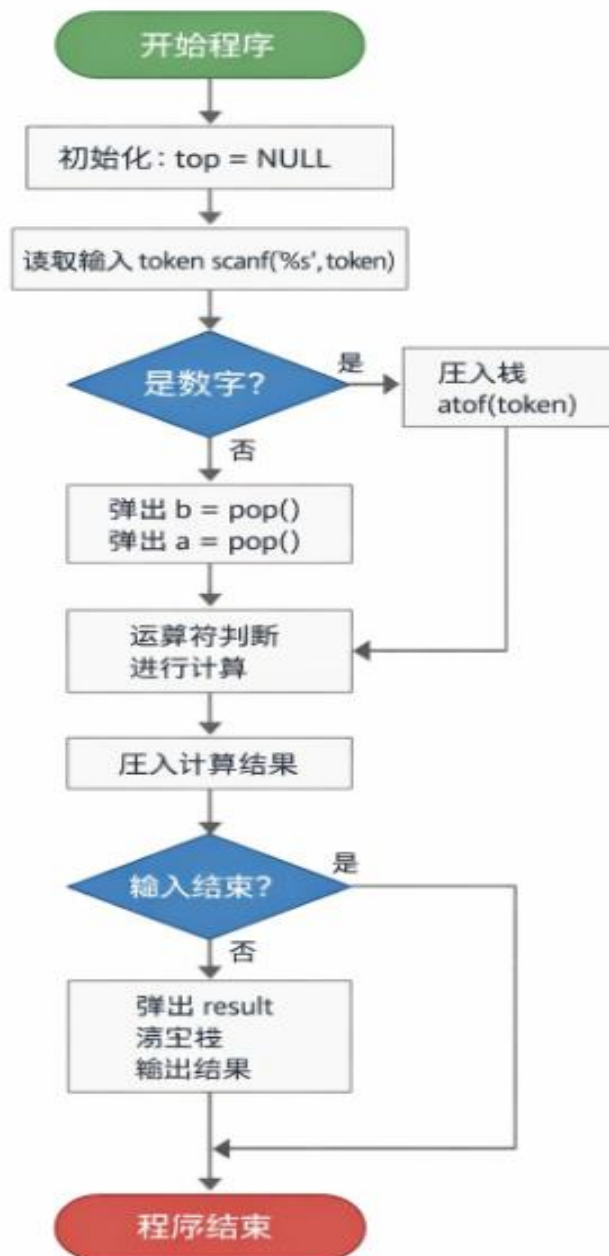


图 2-3 程序设计题 2.3.5 的流程图

我的源代码如下:

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

typedef struct StackNode {
    double value;
    struct StackNode *next;
} StackNode;
  
```



```

StackNode *top = NULL;

void push(double val) {
    StackNode *new_node = (StackNode *)malloc(sizeof(StackNode));
    new_node->value = val;
    new_node->next = top;
    top = new_node;
}

double pop() {
    StackNode *tmp = top;
    double val = tmp->value;
    top = top->next;
    free(tmp);
    return val;
}

double evaluate_rpn() {
    char token[20];
    while (scanf("%s", token) != EOF) {
        if (isdigit(token[0]) || (token[0] == '-' && isdigit(token[1])) || (token[0] == '.' && isdigit(token[1])) || (token[0] == '-' && token[1] == '.' && isdigit(token[2]))) {
            push(atof(token));
        } else {
            double b = pop();
            double a = pop();
            double res;
            switch (token[0]) {
                case '+': res = a + b; break;
                case '-': res = a - b; break;
                case '*': res = a * b; break;
                case '/':
                    if (b == 0) {
                        printf("除数不能为0! \n");
                        exit(1);
                    }
                    res = (int)a / (int)b; // 修正: 强制整数除法
                    break;
                default:
                    printf("无效运算符! \n");
                    exit(1);
            }
            push(res);
        }
    }
}

```

```
    double result = pop();
    while (top != NULL) pop();
    return result;
}

int main() {
    double result = evaluate_rpn();
    if (result == (int)result) {
        printf("%d\n", (int)result);
    } else {
        printf("%f\n", result);
    }
    return 0;
}
```

2.4 小结

通过本章验，我真正明白了如何把结构体作为链表节点的数据容器，也更加了解通过分配动态内存管理的链表操作方法。在实践中，我巩固了指针与结构联用的语法，亲手实现了链表创建、遍历、排序、查找等核心功能。同时我也深刻理解了内存申请与释放的成对管理原则。

我的程序调试能力在解决各类问题的过程中得到显著提升。比如编写学生链表排序功能时，因为交换指针域时没有关注前驱节点，运行失败，后续我通过手绘制链表结构示意图一步步推演运行步骤并修复了问题；在逆波兰表达式计算程序中，因为释放节点后未将指针清除，程序频繁崩溃，排查许久才发现是指针变成悬挂指针。这些经历跟踪数据状态变化，逐步定位问题根源。

不过我也清醒认识到自身的不足。首先，面对复杂数据关系时设计能力不足。然后，程序完整性比较差，没有充分使用场景，比如学生成绩录入时没有检查负数等；最后算法掌握的很少，排序仅采用效率一般的冒泡排序，代码仍有很大优化空间。

这次实验让我深刻体会到，编程绝非理清逻辑就能顺利完成，从构思到落地的每一个细节都至关重要。结构体让零散的数据形成统一整体，链表则让这些数据节点产生关联。每个节点从 malloc 创建到 free 销毁，每一次指针域的调整，只要出现一丝疏漏，就可能导致整个程序崩溃。从最初写代码时的思路混乱，到调试时的反复碰壁，再到最终程序成功运行，这种从困境到通畅的过程，是单纯学习理论无法体会的。

这段经历不仅让我对计算思维有了更具体的认知，更直观理解了程序在计算机中的运行过程：指针如何移动、内存如何分配、数据如何传递。这些实践经验远比课本知识更加深刻。虽然 C 语言课程即将结束，但我深知自己需要学习的内容还有很多，后续会继续钻研更高效的算法，注重程序完整性的设计，将本次实验的收获转化为后续学习的坚实基础。

参考文献

- [1] 卢萍, 李开, 王多强, 甘早斌. C 语言程序设计典型题解与实验指导, 北京: 清华大学出版社, 2019
- [2] 卢萍, 李开, 王多强, 甘早斌. C 语言程序设计, 北京: 清华大学出版社, 2021
- [3] 爱编程的魏校长. C 语言从入门到精通, 北京: 化学工业出版社, 2020